

Medical Triage NLP

Cenário do Problema, Arquitetura e Justificativa Técnica

Autor: Gabriel Pereira Costa

1. Contexto do Problema

O projeto Medical Triage NLP foi desenvolvido a partir do cenário do Tech Challenge: um hospital de referência necessita de uma solução de triagem automática de textos médicos, com foco em processamento rápido e suporte à classificação. O desafio técnico envolve não apenas o modelo, mas também seu ciclo de vida em produção: API REST, Docker, CI/CD, Airflow, monitoramento e otimização de latência.

2. Desafio

- Classificar textos médicos automaticamente utilizando NLP.
- Disponibilizar o classificador por uma API REST.
- Manter baixa latência na inferência.
- Empacotar o serviço em Docker.
- Automatizar lint, testes e build.
- Possuir um processo funcional de treinamento/retreinamento com Airflow.
- Monitorar requisições, latência e erros com Prometheus e Grafana.
- Aplicar uma técnica de otimização de performance.
- Medir baseline, latência otimizada e throughput.

3. Objetivo da Solução

Construir uma solução leve de classificação de textos médicos, disponibilizada como serviço REST, com arquitetura preparada para evolução em cloud e com práticas de produção: CI/CD, orquestração de treinamento, observabilidade, containerização e otimização da inferência.

4. Decisão Arquitetural

O cenário descrito possui característica de inferência real-time: o texto é enviado à API e a classificação deve ser retornada imediatamente. Por isso, foi adotado um serviço FastAPI containerizado. A separação entre API, treinamento, modelo, testes e monitoramento reduz acoplamento e facilita manutenção e evolução.

Medical Triage NLP

Cenário do Problema, Arquitetura e Justificativa Técnica

Autor: Gabriel Pereira Costa

5. Estrutura do Projeto e Justificativa

A estrutura foi organizada para separar responsabilidades e tornar o projeto reproduzível:

```
app/ → API e inferência
training/ → treinamento, conversão e benchmarks
airflow/dags/ → orquestração do treinamento
tests/ → testes automatizados
model/ → artefatos do modelo
data/ → datasets
monitoring/ → configuração Prometheus
.github/workflows → CI/CD
Dockerfile → imagem da API
docker-compose.yml → stack de serviços
```

- **app/**: concentra o serviço executado pela API, separando inferência do treinamento.
- **training/**: reúne treinamento, conversão para ONNX e medições de performance.
- **tests/**: permite validação automatizada independente da aplicação.
- **airflow/dags/**: mantém a orquestração separada do serviço.
- **monitoring/**: isola a configuração do Prometheus.
- **model/**: centraliza os artefatos gerados pelo treinamento.

6. Justificativa do Modelo

Foi escolhido o pipeline **TF-IDF + Logistic Regression**. O enunciado solicita um classificador de texto NLP leve e apresenta TF-IDF com modelos leves como referência. A solução adotada atende a esse objetivo e permite medir de forma objetiva o ganho de inferência após a conversão para ONNX Runtime.

TF-IDF

O TF-IDF transforma o texto em uma representação numérica esparsa baseada na importância dos termos. A implementação utiliza até 10.000 características, unigramas e bigramas e transformação sublinear.

Logistic Regression

A Logistic Regression é um classificador supervisionado leve e adequado para classificação multiclasse sobre representações numéricas de texto. Seu baixo custo computacional é coerente com o requisito de baixa latência.

Justificativa da simplicidade

Um modelo mais pesado aumentaria a complexidade de infraestrutura sem ser necessário para demonstrar os requisitos centrais da fase. A escolha atual permite concentrar o trabalho em deploy, observabilidade, CI/CD, Airflow e otimização de inferência.

Medical Triage NLP

Cenário do Problema, Arquitetura e Justificativa Técnica

Autor: Gabriel Pereira Costa

7. Dataset e Limitações

Foi utilizado o Medical Abstracts TC Corpus. O treinamento possui 11.550 registros e o teste 2.888 registros, com texto médico e rótulo de condição. O arquivo de labels associa cinco categorias de condições médicas.

Existe uma limitação importante de aderência ao cenário clínico: as cinco categorias do dataset não representam diretamente as classes de urgência normal / atenção / urgente do enunciado. Portanto, o projeto demonstra o pipeline técnico de classificação de texto médico, mas o modelo atual não deve ser apresentado como um classificador clínico de urgência. Para isso seria necessário um dataset especificamente rotulado por prioridade clínica.

8. Treinamento e Avaliação

O script training/train.py lê os dados, valida as colunas, treina TF-IDF + Logistic Regression, avalia no conjunto de teste e salva model.pkl. A DAG Airflow reproduz o fluxo de leitura/validação e treinamento.

- Accuracy no conjunto de teste: 0,5471.
- Classification report e matriz de confusão são gerados.
- O resultado é utilizado como baseline do classificador.
- Os resultados não devem ser interpretados como métrica clínica.

9. API e Inferência

- POST /predict recebe o texto e retorna classe, nome da classe e latência.
- GET /health verifica a disponibilidade.
- /metrics expõe métricas Prometheus.
- Swagger/OpenAPI permite testar os endpoints.

10. Requisitos Funcionais

- RF01 – Receber texto médico por API REST.
- RF02 – Executar classificação automática.
- RF03 – Retornar a classe prevista.
- RF04 – Retornar o nome da classe.
- RF05 – Informar a latência.
- RF06 – Disponibilizar health check.
- RF07 – Expor métricas.
- RF08 – Treinar e salvar o modelo.
- RF09 – Orquestrar treinamento por Airflow.
- RF10 – Executar lint e testes automaticamente.
- RF11 – Construir a imagem Docker.
- RF12 – Executar inferência otimizada com ONNX Runtime.
- RF13 – Medir latência e throughput.

Medical Triage NLP

Cenário do Problema, Arquitetura e Justificativa Técnica

Autor: Gabriel Pereira Costa

11. Requisitos do Tech Challenge × O que foi Construído

Requisitos obrigatórios

- **Repositório GitHub:** código, configuração e documentação versionados.
- **CI/CD GitHub Actions:** workflow com lint → train/test → Docker build.
- **Airflow:** DAG funcional com leitura/validação do CSV e treinamento/salvamento.
- **Dockerfile:** imagem funcional para FastAPI.
- **Docker Compose:** API + Prometheus + Grafana + Airflow.
- **Histórico semântico:** commits organizados por finalidade técnica.

Bibliotecas requeridas

- **Scikit-Learn:** TF-IDF + Logistic Regression.
- **FastAPI:** API REST.
- **prometheus-client:** métricas.
- **Airflow:** orquestração.
- **ONNX / ONNX Runtime:** otimização.
- **pytest:** testes.
- **Ruff:** lint.

Boas práticas obrigatórias

- **2+ automações:** lint e testes; build Docker encadeado após testes.
- **DAG funcional:** leitura/validação → treino.
- **Grafana:** dashboard com foco em requisições, latência e erros.
- **Otimização:** conversão para ONNX e inferência com ONNX Runtime.

12. Etapa 1 — Decisão Arquitetural e API

Foi adotada uma estratégia de inferência real-time por API. FastAPI recebe o texto, executa o classificador e devolve a resposta. O Dockerfile permite executar o mesmo serviço de forma isolada e reproduzível. O baseline de latência foi medido antes da otimização.

13. Etapa 2 — CI/CD e Treino

O workflow GitHub Actions executa lint, depois treinamento e testes, e por fim build Docker. A DAG Airflow possui tarefas `read_dataset` e `train_model`, com dependência explícita entre elas.

Medical Triage NLP

Cenário do Problema, Arquitetura e Justificativa Técnica

Autor: Gabriel Pereira Costa

14. Etapa 3 — Monitoramento e Observabilidade

A API utiliza Counter para contagem de requisições e Histogram para duração das requisições. O Prometheus coleta /metrics e o Grafana consulta o Prometheus como fonte de dados. O Docker Compose integra os serviços em uma rede comum.

Painéis do dashboard

- Total de requisições/predições.
- Latência/tempo médio de resposta.
- Taxa de erros HTTP.

15. Etapa 4 — Otimização

O pipeline Scikit-Learn foi convertido para ONNX utilizando skl2onnx. O arquivo model.onnx foi validado com onnx.checker. O ONNX Runtime foi configurado com CPUExecutionProvider e utilizado para executar a inferência.

Validação de equivalência

Para o texto de teste utilizado, o modelo original retornou classe 4 e o ONNX Runtime também retornou classe 4. Isso demonstra que a conversão preservou a previsão no caso validado.

16. Benchmark de Latência

Métrica	Scikit-Learn	ONNX Runtime
Latência média	0,5188 ms	0,1042 ms
Diferença	—	0,4146 ms
Redução observada	—	79,91%

17. Benchmark de Throughput

Métrica	Scikit-Learn	ONNX Runtime
Inferências	10.000	10.000
Tempo total	5,0307 s	1,3338 s
Throughput	1.987,79/s	7.497,23/s
Aumento observado	—	277,16%

Os benchmarks representam medições do ambiente local de desenvolvimento. São evidências experimentais da otimização e não garantem a mesma performance em produção.

Medical Triage NLP

Cenário do Problema, Arquitetura e Justificativa Técnica

Autor: Gabriel Pereira Costa

18. Critérios de Avaliação × Evidências

- **Modelagem e Otimização — 20%:** modelo NLP funcional, conversão ONNX e melhoria de latência demonstrada.
- **CI/CD — 15%:** workflow com lint, testes e build.
- **Orquestração Airflow — 15%:** DAG funcional de leitura e treino.
- **Monitoramento — 20%:** Compose com API, Prometheus e Grafana; dashboard para métricas.
- **Documentação — 15%:** arquitetura, justificativas e instruções.
- **Vídeo STAR — 15%:** deve demonstrar Situation, Task, Action e Result em até 5 minutos.

19. Requisitos Não Funcionais

- Desempenho: baixa latência e throughput medidos.
- Observabilidade: métricas expostas e coletadas.
- Confiabilidade: testes automatizados.
- Manutenibilidade: organização modular.
- Portabilidade: Docker.
- Reprodutibilidade: dependências versionadas.
- Automação: GitHub Actions e Airflow.
- Escalabilidade: arquitetura preparada para evolução cloud.
- Documentação: decisões e resultados registrados.

20. Resultado Consolidado

- API FastAPI funcional.
- Dockerfile funcional.
- GitHub Actions com lint, testes e build.
- DAG Airflow funcional.
- Prometheus configurado.
- Grafana conectado ao Prometheus.
- Modelo TF-IDF + Logistic Regression treinado.
- Modelo convertido e validado em ONNX.
- ONNX Runtime executando inferência.
- Baseline, benchmark de latência e benchmark de throughput realizados.

21. Conclusão Técnica

A escolha do TF-IDF + Logistic Regression foi orientada pelo requisito de um classificador NLP leve e pela necessidade de baixa latência. A organização modular separa treinamento, inferência, testes, orquestração e observabilidade. A conversão para ONNX Runtime apresentou, no benchmark local, redução de 79,91% na latência média e aumento de 277,16% no throughput.

A principal limitação de domínio é o dataset: suas cinco categorias são condições médicas, não níveis de urgência. Assim, o projeto demonstra a arquitetura e o pipeline técnico solicitados, enquanto uma aplicação clínica real de triagem de urgência exigiria dados especificamente rotulados para prioridade clínica.

Medical Triage NLP

Cenário do Problema, Arquitetura e Justificativa Técnica

Autor: Gabriel Pereira Costa

22. Roteiro STAR para o Vídeo

- **Situation:** apresentar o problema hospitalar e a necessidade de processar textos rapidamente.
- **Task:** apresentar os requisitos de latência, CI/CD, Airflow e monitoramento.
- **Action:** mostrar arquitetura, modelo, Docker, GitHub Actions, Airflow, Prometheus/Grafana e ONNX Runtime.
- **Result:** demonstrar API e resultados: latência 0,5188 ms → 0,1042 ms e throughput 1.987,79/s → 7.497,23/s.

23. Artefatos e Execução

- API e código de treinamento.
- Workflow GitHub Actions.
- DAG Airflow.
- Dockerfile e Docker Compose.
- Configuração Prometheus.
- Dashboard Grafana.
- model.pkl e model.onnx.
- Scripts de benchmark.
- README com arquitetura e resultados.
- Vídeo final no formato STAR.

24. Próximas Evoluções Técnicas

- Integrar definitivamente o ONNX Runtime ao app/model.py da API de produção.
- Gerar o model.onnx automaticamente no pipeline antes do Docker build.
- Executar benchmark também através do endpoint HTTP para medir o impacto do overhead da API.
- Ampliar o dashboard com métricas adicionais, se necessário.
- Considerar dataset especificamente rotulado para urgência clínica em uma evolução futura.
- Documentar a estratégia de deploy cloud para real-time conforme solicitado na Etapa 1.

25. Encerramento

A solução consolida as disciplinas da fase em um fluxo único: dados → treinamento → modelo → API → Docker → CI/CD → Airflow → monitoramento → otimização. A documentação deve ser utilizada em conjunto com o README, o repositório e a demonstração em vídeo.